CA 2: Machine Learning Assessment Short Report

Student Name: Marek Martinak

Student Number: D00250465

Due Date: 30/03/2026

Dataset: https://www.cs.toronto.edu/~kriz/cifar.html

GitHub Link Code: https://github.com/Aang1214/MachineLearning_CA2.git

0. **Step: SELECTING DATASET**

The selected dataset was chosen based on several factors. Techniques, which will be most likely used, can be implemented in real-life scenarios such as self-driving cars. It is a valuable learning point to gain, where the experience could also be used to help identify diseases in crops or X-ray images. There is a lot of help on the internet, which can be useful to beginners. It is free and with no ethical concerns, and that is perfect for this assignment.

It is a dataset that consists of 60,000 32x32 color images in 10 different classes, with 6,000 images per class. There are classes such as dog, cat, bird, or also something like truck and ship.

1. **Step: IMPORT LIBRARIES**

First step was to load all necessary libraries, which will be used during the project.

2. **Step: LOAD CIFAR-10 DATASET**

This is the step where it is possible to learn more about the dataset and if it contains the same information from the website. As being said, each class contains 6,000 images. Therefore, this dataset is completely balance and perfectly prepared for machine learning model. We were able to define all classes such as:

- airplane
- automobile
- bird
- cat

- deer
- dog
- frog
- horse
- ship
- truck

Dataset was load and was already split between two groups. First is training data, which contains 50,000 images. The second is testing data, which contains 10,000 images.

3. Step: DATA PREPROCESSING

Pixel value was changed from 0-255 to 0-1 and converted the float type from 64 to 32. It was necessary step for training stability model performance and memory usage. Creating binary vector via one-hot encoding is essential for further the categorical cross entropy loss function. It also allows to eliminate possible false ordinal relationship such as, cat(4) is lower than deer(5).

4. Step: CREATE DATA

train_dataGenerator contains augmentation techniques such as shear, zoom, width/height shifts, horizontal flips, and rotationcontains were chosen based on actively trying to find the best configuration. This version serves to prevent overfitting. On the other hand, the validation generator only applies rescaling, because augmentation would mask the true performance. The batch size was set to be 64 - to have a more balanced memory and proper training speed. Flow serves as a bridge between in-memory data and the ImageDataGenerator. It also shuffles the training dataset to prevent possible bias and learns the actual pattern. Lastly, to complete one epoch, it is necessary to calculate the steps properly, meaning how many batches to process.

5. Step: BUILD CNN MODEL

For the CNN model, it was necessary for the model to be sequential, which means it is linear and each layer feeds directly into the next. Input shape was set up to be (32, 32, 3) because the images have the same values. There are 5 Convolutional Layers. The difference between them is the increasing filter count from 32 to 256. It can then receive complex patterns. MaxPooling was used in some layers to reduce spatial size. In almost every layer, we used Batch Normalization to make training faster and more stable. To be able to change 2D feature maps to a 1D vector, the flatten layer was inserted into it. It is used mainly for Dense layers, which allow neurons to connect with one another.

Overfitting is one of the factors to watch out for, which is why the dropout method was implemented, allowing for the random removal of neurons. Lastly, Softmax was used for multi-class classification. It converts numbers into probabilities.

6. Step: COMPILE MODEL

Adam optimizer was selected with a learning rate of 0.001 for model compilation. It should be able to control the speed, direction, and method of learning from errors. Categorical_crossentropy was ideal to choose for the loss function, because it can measure wrong predictions for multiple classes. The main progress indicator is accuracy, which will be able to show the correct predictions in percentage.

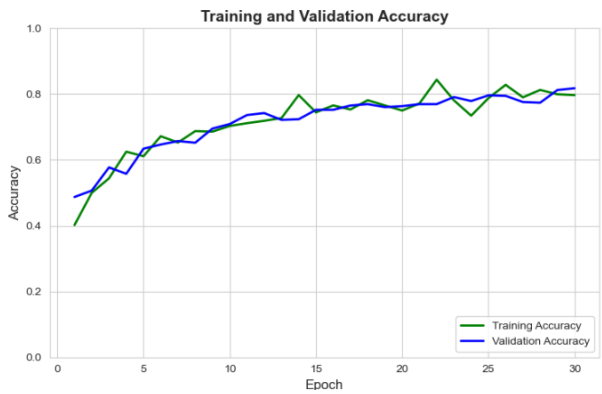
7. Step: SET UP CALLBACKS

Two callbacks were created to train the model. The first is focusing on saving the model's weight when the accuracy improves. The second callback focuses on stopping the model, when it is not improving. It calls this callback, which will stop the training and save the best performance.

8. Step: TRAIN MODEL

The training model was set up to complete 30 epochs. Now, it complete each epoch. There were moments when the training model stopped the training before each epoch was completed, because it was not improving. This means that callbacks are functional, but they were not used in this case.

9. Step: PLOT TRAINING RESULTS



The best validation accuracy number is 0.8178 at epoch 30, and the best training accuracy number is 0.7969. Two plots were created to show how the model was improving at each epoch.

Left plot:

Two lines are close to each other. The training accuracy ranged from 0.40 to 0.80, and the validation accuracy ranged from 0.48 to 0.82. After epoch 20, there is a slight improvement.

Right plot:

The right plot shows two curves with training and validation loss. Almost with each epoch is decreasing, which is a good sign of having no overfitting. The small fluctuations are okay and normal.

Overall, the model is learning properly. It confirms there is no overfitting or underfitting. Based on these results, the model is suitable for use in the next steps.

10. Step: EVALUATE MODEL

The model was tested on unseen data. The Test Loss is 0.5367 and the Test Accuracy is 81.79%. This is a good sign of prediction confidence and a well-designed model. The performance confirms similar results with validation accuracy.

11. Step: CLASSIFICATION REPORT

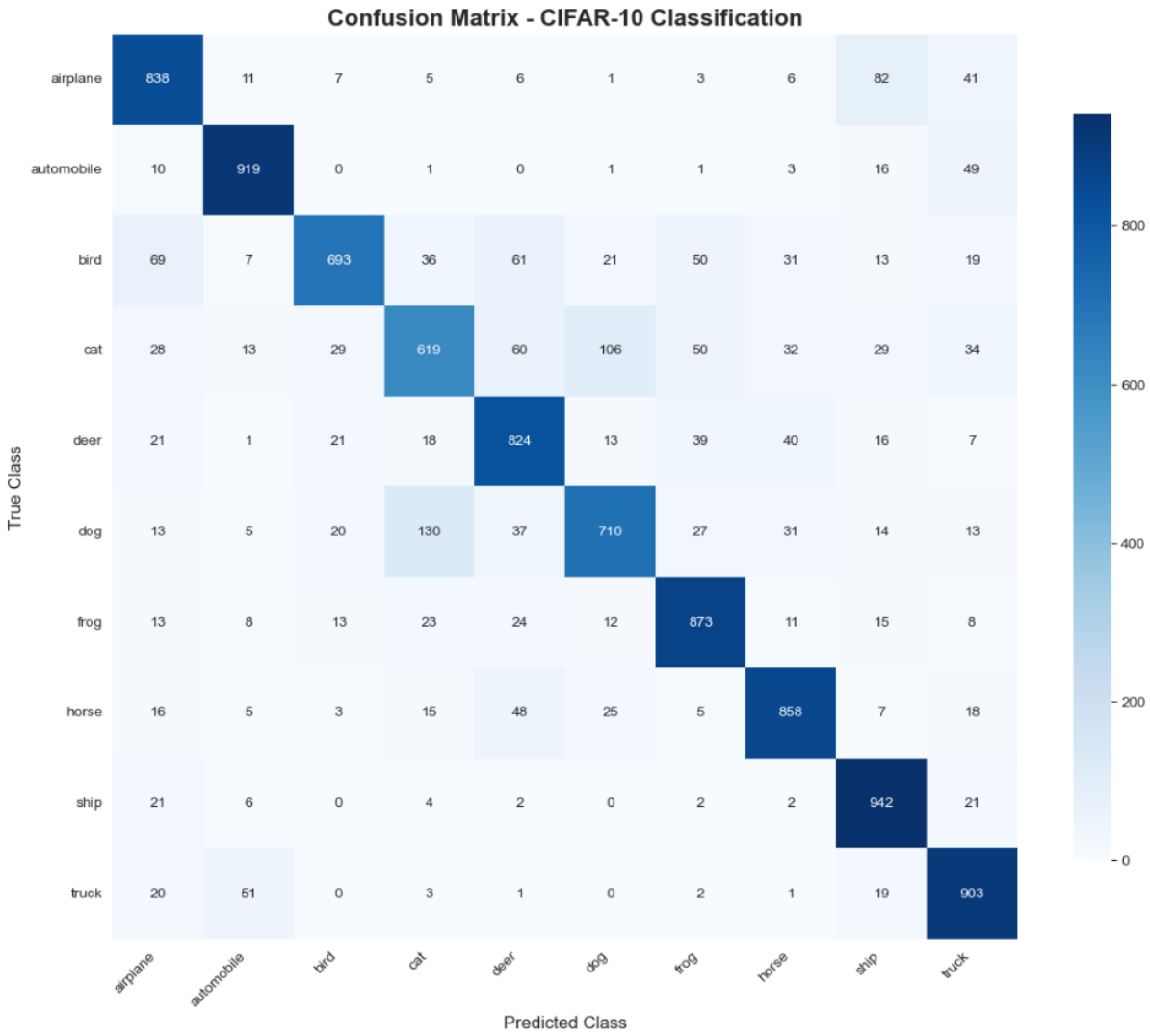
```
Classification Report:
--------------------------------------------------------
              precision    recall  f1-score   support

    airplane       0.80      0.84      0.82      1000
  automobile       0.90      0.92      0.91      1000
        bird       0.88      0.69      0.78      1000
         cat       0.72      0.62      0.67      1000
        deer       0.78      0.82      0.80      1000
         dog       0.80      0.71      0.75      1000
        frog       0.83      0.87      0.85      1000
       horse       0.85      0.86      0.85      1000
        ship       0.82      0.94      0.88      1000
       truck       0.81      0.90      0.85      1000

    accuracy                           0.82     10000
   macro avg       0.82      0.82      0.82     10000
weighted avg       0.82      0.82      0.82     10000
```

The Classification Report showed the worst performing classes are dog, bird, and especially cat. It can be debatable which is the best performing, because the automobile has a precision 90% and a recall 92%. The ship, on the other hand, has a precision 82%, but the recall is amazing 94%. Regardless of choosing which one is better, they are performing well enough to correctly identify images. With 10 000 images, the overall accuracy is 82%.

12. Step: CONFUSION MATRIX



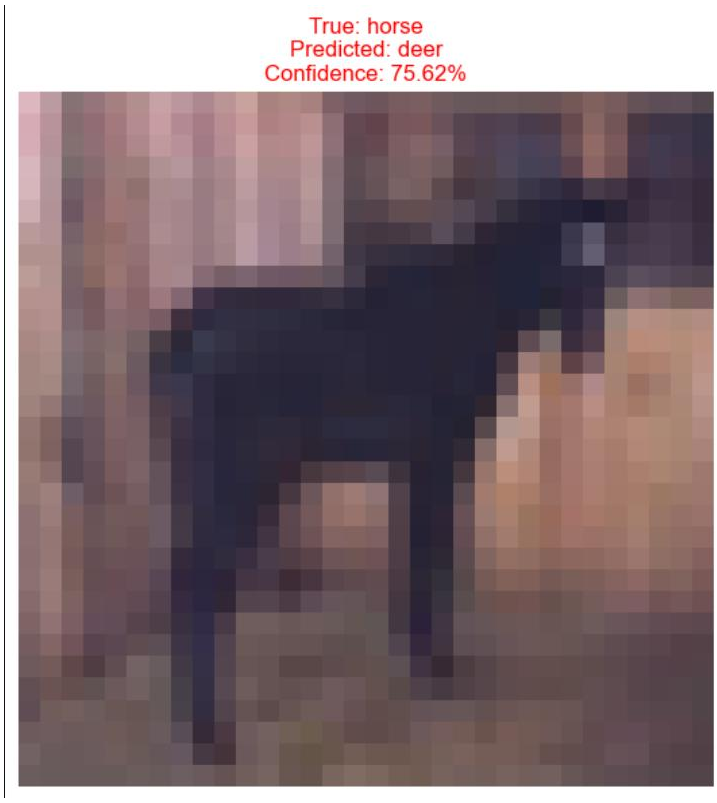
This is a confusion matrix that visually shows the predictions of each class. Most of the class shows high correct predictions (ranging from 700 to 900). High prediction accuracy comes from classes such as ship, automobile, truck, frog, and horse. Misclassification concerns mainly the bird, cat, and dog classes.

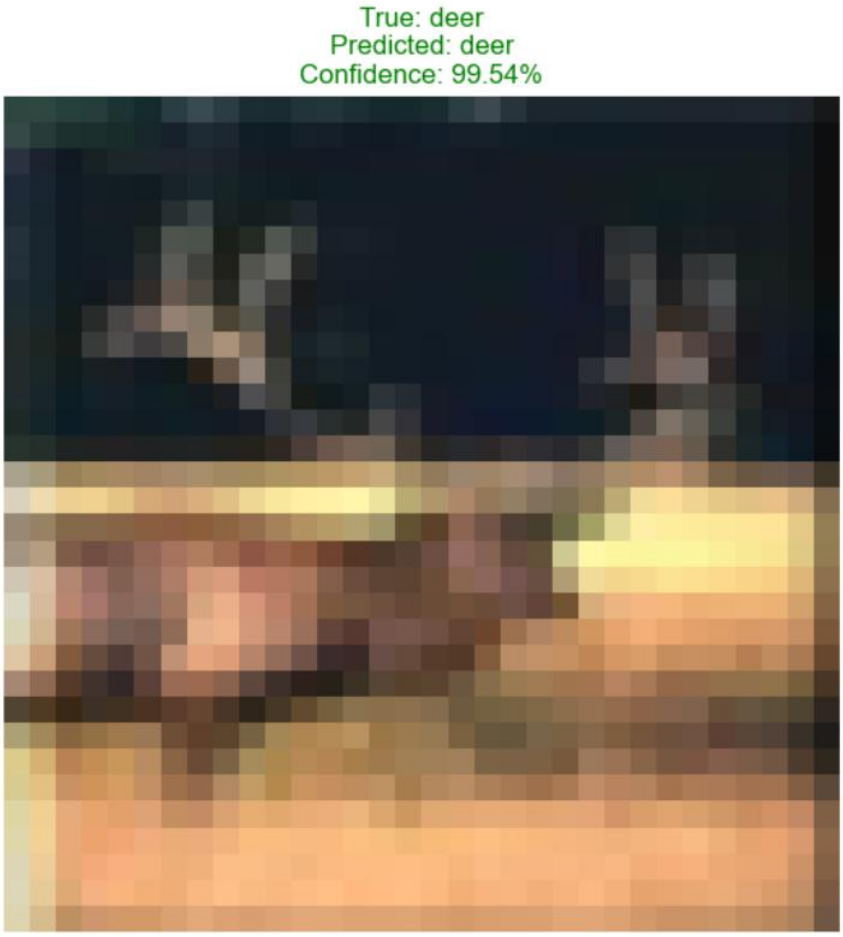
Overall, the model is reliable. It is confused mainly with cat and dog classes. The bird class is confused across all the predictions, but mainly with airplane (69), deer (61), and frog(50). The best predicted class is ship. Errors come from similar-looking objects, which is understandable.

13. Step: PREDICT A SINGLE IMAGE

This step is only to test the model on a single image. It shows the true label, predicted label, and how confident the model is about the image. The image is randomly chosen and later displayed on the screen. If the prediction is incorrect, the color will change to red. Prediction is based on the highest number from the array.

Examples:


True: horse
Predicted: deer
Confidence: 75.62%


True: deer
Predicted: deer
Confidence: 99.54%

14. Step: MODEL IMPROVEMENT ANALYSIS

OVERALL PERFORMANCE:

Test Accuracy: **0.8179 (81.79%)**

Test Loss: **0.5367**

Best Validation Accuracy: **0.8178**

While doing this project, there were a few challenges ahead. A small amount of knowledge about the machine learning principles regarding image classification. Small image size (32x32) limits feature extraction. Model being confused with similar classes. Preventing as much as possible to not overfit the model. Some of these challenges remain even after all this, and it cannot be said with certainty that they will be resolved next time. Still, the project was trying to handle the challenges well.

To improve this model even more. The solution for that could be adding more convolutional layers to the architecture. Another improvement could be adjusting the learning rate or increase dropout number. Lastly, there could be more experiments with data augmentation.